OIM Automation Session

Claude Code session export · project `playwright-real-draft-v3-claude` · 2026-09-23
Tool calls (file reads, builds) are summarized; the messages are reproduced as sent.

USER

I want you to analyze the project. It is an automation playwright and java project separated from any source code or application under test. There are a bunch of set of technologies used into this project structure. I want a detailed report after you complete analyzing it.

CLAUDE

Analysis report: OIM test automation framework (Playwright + Java)

The project is a separate, black-box UI test framework for **MAPFRE Perú's OIM portal** (`oim.pre.mapfre.com.pe`). It contains no application code. It has about 300 source files: 25k lines of page objects and 5k lines of step definitions. I checked that it compiles: `mvn -o test-compile` passes with Java 17.

1. Technology stack

Layer	Technology	Version	Notes
Language / build	Java, Maven	17 / 3.9.7	`maven-compiler-plugin` 3.13, `release=17`
Browser automation	Microsoft Playwright for Java	1.57.0	Chromium, Firefox or WebKit, chosen by setting
BDD	Cucumber (`cucumber-java` , `cucumber-testng`)	7.33.0	Gherkin steps written in Spanish (English `Given/When/Then` keywords)
Test runner	TestNG, run through Surefire 3.5.4	7.11.0	Scenarios run in parallel through `@DataProvider(parallel=true)` threads
Dependency injection	cucumber-picocontainer	7.32	Each scenario gets its own `ScenarioContext` , `PageProvider` , `SchedulingState`
Reporting	ExtentReports 5 (Spark + JSON), Cucumber HTML/JSON/JUnit	5.1.2	JSON output from each pod is merged into one report in CI
Logging	SLF4J + slf4j-simple	2.0.17	Plus a custom console step logger
Test data	Apache POI (xls/xlsx), Gson, commons-io, commons-lang3	5.5.1 …	Excel upload templates are edited at runtime
CI	Jenkins declarative pipeline on Kubernetes pods	—	Shared libraries `global-pipeline-library` , `pe-common-pipeline-config` ; code from Bitbucket; logs to Logstash

2. Architecture

```
src/main/java/com/mapfre       ← reusable framework + page objects
 ├─ config/        ConfigManager (priority: -D > ENV > .properties), EnvironmentConfig (app.
 ├─ playwright/driver/  DriverFactory, DriverManager (ThreadLocal Playwright/Browser/Context
 ├─ playwright/pageobjects/   BasePage (889 lines, ~70 helpers) + ~125 pages by business are
 ├─ playwright/components/    MaterialAutocomplete, MaterialDatePicker, SearchResults compor
 ├─ utils/waits/   FirstAppearanceRace, FirstDisappearanceRace, GestorModal, OptionalModalWa
 ├─ utils/         UiSync (spinner "waves" + quiet period), Checkpoint, DownloadVsErrorRace,
 ├─ asserts/       ElementAsserts (wrapper over PlaywrightAssertions)
 └─ reporting/     ArtifactSink / ArtifactSinks (main code emits screenshots without knowing
src/test/java/com/mapfre
 ├─ runner/        RunnerCucumberTest (AbstractTestNGCucumberTests)
 ├─ hooks/         Hooks (@Before/@AfterStep/@After), ExtentLifecycleHooks, ScenarioContext,
 ├─ reporting/     ExtentCucumberStepsPlugin, ConsoleStepLogger, ExtentReportManager, Extent
 └─ stepdefinitions/  ~85 step classes mirroring the page-object tree
src/test/resources
 ├─ features/      55 .feature files, 81 Scenario Outlines
 ├─ docsFeatures/  22 Markdown docs describing features
 ├─ testdata/uploads/templates/  xls/xlsx/pdf/txt upload templates
 └─ images/        vehicle inspection photos for upload
```

Business areas covered: policy quote and issuance for Auto (new and used, with inspection), SOAT, SCTR (short and regular period), Vida Ley, Accidentes, Salud, Vida Individual, Vida Inversión, FOLA, Decesos, Camposanto, Transporte and Riesgos Generales; the *Constancias SCTR / SCTR Minería / Vida Ley* module (declaración, inclusión, manual, reemplazo, reimpresión, anulación, mes vencido); and *Inspección de Autos*.

How one scenario runs

1. The TestNG data provider hands scenarios to threads (`dp.threads` , default 1; Jenkins uses 2).
2. `Hooks.@Before` sets up the screenshot sink, then `DriverFactory.init()` creates a full Playwright instance, browser, context and page for that scenario, stored per thread, and creates the Extent test.
3. Step classes get a `PageProvider` and a `ScenarioContext` through PicoContainer and build their page objects in the constructor.
4. Page methods follow an *action* + `UiSync.waitForAppIdle` pattern (spinner hidden and stays hidden for 250 ms). Harder transitions use race utilities such as `FirstAppearanceRace` (error vs success vs timeout).
5. `@After` takes a screenshot on failure, then `DriverManager.cleanup()` and clears per-thread state.

CI pipeline (`JenkinsCI`)

1. **Discover:** Cucumber dry run with the chosen tags → JSON → list of `feature:line` entries.
2. **Split:** list cut into groups of 2, each group gets its own Kubernetes pod, all in parallel; each pod runs 2 threads.
3. **Per pod:** runs tests, strips `- #1` suffixes from JUnit names (for Octane), publishes JUnit, stashes `extent.json` .
4. **Merge:** `ExtentJsonMergeMain` via `exec:java` merges all pods' JSON into one Spark HTML report, published with `publishHTML` and archived.

3. Strengths

- Clean layering; `ArtifactSink` stops `src/main` from depending on Cucumber.
- Built for parallel runs: per-thread driver and Extent state, PicoContainer-scoped scenario state.
- Modern locators: ~580 `getByRole` , 195 `getByText` , 83 `getByLabel` vs 27 XPath.

- App-aware synchronization (spinner waves + quiet period, first-appearance races).
- 12-factor configuration: any key can be overridden with `-D` or an environment variable.
- Rich evidence: screenshots per step node, named checkpoints, readable console log with timings.
- Living docs in `docsFeatures/*.md`.

4. Findings and risks

🔴 High

1. **Plain-text credentials and an internal URL in the repo** (`user.email=WEBMASTER`, `user.password=…` in `global.properties`). `LoginPage` read them via `ConfigManager.getProperty`, which skips the `-D`/ENV override. Rotate the password.
2. **`RandomData` returns the same values every time.** `rnd()` builds `new Random(SeedManager.seed())` on every call with the same per-thread seed, and `SeedManager.clear()` is never called — plate/VIN/engine repeat across scenarios on the same thread.
3. **The runner hardcodes `tags = "@riesgosgenerales"`.**

🟠 Medium

4. Two screenshots per step (`@AfterStep` + `ExtentCucumberStepsPlugin`).
5. A full Playwright instance and browser start per scenario (~1–2 s each).
6. `DriverManager.cleanup()` is not exception-safe.
7. `ElementAsserts` discards the original error (no cause).
8. Fixed waits: 54 `waitForTimeout`, 2 `Thread.sleep`, `waitRandomBetween`, JS clicks in `clickMenuOption`.
9. JavaScript built by string concatenation in `clickMenuOption`.
10. 143 broad `catch (Exception …)` and 73 `System.out.println`.

🟡 Low / maintainability

11. Very long steps (up to 16 parameters); all 81 scenarios are single-row Scenario Outlines; no `Background`.
12. `BasePage` is an 889-line catch-all with near-duplicate helpers and mixed-language names.
13. Inconsistent tags (`@regresion` / `@Regresion`) and typos in folder/step names; leftover files.
14. Placeholder Maven metadata (`org.example`), no Maven Wrapper, saucedemo URLs, unused `implicitWait`.
15. Hardcoded relative paths in `TemplateFactory`.
16. Reporting split across `src/main` and `src/test`; double flush.
17. No unit tests for utilities, no static analysis, no README.

5. Recommended next steps

1. Remove the credentials, rotate the password, switch `LoginPage` to `getGlobal`.
2. Fix `RandomData` / `SeedManager`.
3. Remove the hardcoded runner tag.
4. One screenshot per step; safer `cleanup()` and `ElementAsserts`.
5. One browser per thread, one context per scenario; state-based waits.
6. Gradual refactor: `Background` for login, shorter steps, split `BasePage`, one tag convention.

USER

let start fixing the top issues you categorized as High

CLAUDE

(Plan mode.) Asked three questions:

Question	User's answer
Default tags for a plain `mvn test` once `@riesgosgenerales` is removed?	`"not @wip"`
How should the pipeline supply the OIM login?	Don't touch JenkinsCI
How to supply credentials locally?	Git-ignored local file `config/local/global.properties`

The plan was approved, then implemented.

I fixed all three High issues. The project compiles and every check passed.

> ⚠️ **Your Jenkins runs will fail at login until you add the credentials there.** The login no longer lives in the repo and `JenkinsCI` was not touched. Provide `USER_EMAIL` and `USER_PASSWORD` to the `mvn test` step in the pod stage (e.g. `withCredentials`). Also **rotate the `WEBMASTER` password**: it is still in Bitbucket history, and in `target/classes` until the next clean build.

What changed

1. Credentials

- `global.properties`: username and password removed; a comment explains the three ways to supply them.
- `ConfigManager`: also reads an optional `config/local/global.properties`. Priority: `-D` → environment variable → local file → repo file.
- `LoginPage`: reads credentials through the `-D`/ENV-aware lookup; fails with a clear message if one is missing.

2. Repeating test data

- `SeedManager`: one random sequence per scenario; `clear()` resets it.
- `RandomData`: `numeric`, `fullName`, `uniqueSuffix` return a new value per call. Plate, VIN and engine number still intentionally share their first 6 characters within a scenario.
- `Hooks`: new seed per scenario, cleared at the end; seed logged to Extent (reproduce with `-Ddata.seed=<n>`).

3. Runner default tag

- `RunnerCucumberTest`: default is now `"not @wip"`; `-Dcucumber.filter.tags` still overrides it.

Checks I ran

- `mvn test-compile` succeeds.
- Dry run with no tags finds all **81** scenarios; with `@riesgosgenerales` it finds 7.

- Verification program: `numeric()` values differ; plate changes between scenarios; prefix still shared; `data.seed=42` reproduces values; credentials read from env vars and local file, env var wins.
- No password remains under `src/`.

A real login against `oim.pre.mapfre.com.pe` was not run.

Existing bug that the default run now shows

Scenario 37 in `oim_emision_auto.feature` (line 14, "Emisión Póliza Auto Nuevo ... Datos del Vehiculo ingresamos la frecuencia de uso...") uses a step with no matching step definition. It was already broken; the old hardcoded tag hid it.

USER

where and how should I credentials in config/local/global.properties?

CLAUDE

Create the file at the **project root**, next to `pom.xml`, not under `src/`:

```
D:\workspace\playwright-real-draft-v3-claude\
├─ pom.xml
├─ JenkinsCI
├─ config\
│  └─ local\
│     └─ global.properties   ← create this
└─ src\...
```

Contents: only the keys you want to override.

```
user.email=WEBMASTER
user.password=YourNewPasswordHere
```

To create it from PowerShell in the project root:

```
New-Item -ItemType Directory -Force config\local
@'
user.email=WEBMASTER
user.password=YourNewPasswordHere
'@ | Out-File -Encoding utf8 config\local\global.properties
```

- **Working directory:** the path is resolved from where tests run. `mvn test` from the project root works; in IntelliJ the run configuration's working directory must be the project root.
- **Format:** Java `.properties`, read as UTF-8. Write a backslash as `\\`; trailing spaces are kept.
- **Priority:** `-Duser.password=...` or `USER_PASSWORD` replaces the file value.

- **Git:** `config/local/` is in `.gitignore`.
- **Other settings:** the file can override any other `global.properties` key for your machine only.
- **Password:** use the rotated password.

USER

I still cannot understand. Where the user and password would come from to run tests in jenkins if config/local/global.properties is the ingore file? When you said "the file can override any other global.properties key for your machine only, such as env=qa or ui.spinner.timeoutMs" that means my src/resources/global.properties is useless? When you should I write \\ symbol, give me an example.

CLAUDE

1. Where Jenkins gets the user and password

Jenkins never sees `config/local/global.properties`; that file lives only on your laptop. Jenkins uses **environment variables**, which `ConfigManager` checks before any file:

Key the code asks for	Environment variable it checks
`user.email`	`USER_EMAIL`
`user.password`	`USER_PASSWORD`

Step 1: create the credential in Jenkins (*Manage Jenkins → Credentials → Add Credentials*): Kind = Username with password; Username = `WEBMASTER`; Password = the new password; ID = `oim-qa-login`.

Step 2: wrap the `mvn test` in the pod stage with `withCredentials`:

```
catchError(buildResult: 'UNSTABLE', stageResult: 'UNSTABLE') {
  withCredentials([usernamePassword(credentialsId: 'oim-qa-login',
                                    usernameVariable: 'USER_EMAIL',
                                    passwordVariable: 'USER_PASSWORD')]) {
    sh """
      mvn -B -q test \
        ...   (same flags as today)
    """
  }
}
```

Jenkins sets the two variables and masks the password in the log. The Discover and Merge stages don't log in and don't need it.

- **Your laptop:** `config/local/global.properties` (or environment variables).
- **Jenkins:** Jenkins credential store → `USER_EMAIL` / `USER_PASSWORD`.
- **The repo:** nowhere.

2. `src/main/resources/global.properties` is still needed

It is the main settings file everyone uses, including Jenkins. The local file only overrides the keys written in it. Example with a local file containing only the two credentials:

Key	Value used	Comes from
`env`	`qa`	`src/main/resources/global.properties`
`app.url.qa`	`https://oim.pre.mapfre.com.pe/#/login`	`src/main/resources/global.properties`
`ui.spinner.timeoutMs`	`120000`	`src/main/resources/global.properties`
`user.email`	`WEBMASTER`	your local file
`user.password`	`Secret123`	your local file

3. When to write `\\`

Only if your password contains a backslash. For a real password `Mapfre\2025`:

What you write in the file	What the code receives	Correct?
`user.password=Mapfre\2025`	`Mapfre2025`	❌ login fails
`user.password=Mapfre\\2025`	`Mapfre\2025`	✅

- A leading space: `user.password=\ abc` gives `abc`.
- A trailing backslash (`abc\`): write `abc\\`.
- `.` `!` `@` `#` `$` `%` `=` `:` in the middle need no escaping.
- Environment variables in Jenkins take the value exactly as typed.

USER

can export this whole chat in pdf or html document?

CLAUDE

Exported this conversation to `docs/chat-export.html` and `docs/chat-export.pdf` in the project folder.